starts the test runner, and which is why our test files have been executing tests when invoked just as plain Ruby files.

Unit tests are often combined into high-level groupings, called *test cases*. The test cases generally contain all the tests relating to a particular facility or feature—in Ruby, often each application class will have one associated test case. Within the test case, you'll typically want to organize your assertions into separate test methods, where each method contains the assertions for one type of test; one method could check regular number conversions, another could test error handling, and so on. (We'll see later that RSpec allows you to structure tests a little bit differently.)

The classes that represent test cases must be subclasses of Minitest::Test. The methods that hold the assertions must have names that start with test_. This is important: the testing framework dynamically searches the test methods to find tests to run, and only methods whose names start with test_ are eligible.

Quite often you'll find all the test methods within a test case start by setting up a particular scenario. Each test method then probes some aspect of that scenario. Finally, each method may then tidy up after itself. For example, we could be testing a class that extracts jukebox playlists from a database. (The playlist_builder file contains a DBI class that simulates a database connection for our purposes here.)

unittesting/test_playlist_builder1.rb

```
require "minitest/autorun"
require_relative "playlist_builder"

class TestPlaylistBuilder < Minitest::Test
  def test_empty_playlist
    database = DBI.new("DBI:mysql:playlists")
    playlist_builder = PlaylistBuilder.new(database)
    assert_empty(playlist_builder.playlist)
    playlist_builder.close
  end

  def test_artist_playlist
    database = DBI.new("DBI:mysql:playlists")
    playlist_builder = PlaylistBuilder.new(database)
    playlist_builder.include_artist("krauss")
    refute_empty(playlist_builder.playlist, "Playlist shouldn't be empty")
    playlist_builder.playlist.each do |entry|
      assert_match(/krauss/i, entry.artist)
    end
    playlist_builder.close
  end

  def test_title_playlist
    database = DBI.new("DBI:mysql:playlists")
    playlist_builder = PlaylistBuilder.new(database)
    playlist_builder.include_title("midnight")
    refute_empty(playlist_builder.playlist, "Playlist shouldn't be empty")
    playlist_builder.playlist.each do |entry|
      assert_match(/midnight/i, entry.title)
    end
    playlist_builder.close
  end
```